involve tasks such as data cleaning, normalization, feature extraction, and result formatting.

> **DEFINITION** *Feature extraction* is a data processing technique where the machine learning model identifies meaningful patterns, characteristics, or representations and extracts them from raw data. In AI applications, this could mean detecting key phrases in text or shapes in images. Feature extraction begins with the model learning to automatically extract relevant features from the training data. Then, during inference, the model applies its learned feature-extraction capabilities to new data to understand what it is looking at.

API INTEGRATION

Whether you run an LLM locally or use external APIs, generative AI web apps often integrate with external systems to access additional functionalities or data sources. These APIs could provide services such as natural language processing, image recognition, or sentiment analysis, enhancing the application's capabilities.

DEPLOYMENT AND SCALING

Your web apps need to be deployed and scaled to handle increased spikes and to prevent unwanted service downtime. This task requires careful consideration of infrastructure requirements, resource allocation, and load balancing. Deployment tools and techniques ensure that the application can handle varying levels of traffic and usage.

1.2.2 The flow of user interactions

Now that we've explored the key components of an AI-powered app, let's look at how the user, the app, and the model interact. Figure 1.2 illustrates a typical workflow of actions and information, showing that content creation is a collaborative process between a user and an AI web app.

This user–app interaction can be summed up in four major stages:

1. *User input through the UI interface*—The user submits a query or a prompt, initiating the interaction with the UI. This input can be text prompts, images, selections, or any other format supported by the application.
2. *Backend processing*—The app then routes the query through its internal data processing pipeline, which may include cleaning, feature extraction, and preprocessing, and then selects the most suitable model for the query.
3. *Content generation*—The backend system interacts with both internal and external services or APIs to initialize the generation process in the selected model. This model, which has been pretrained on vast amounts of data (and may be further fine-tuned for specific applications like RAG), then uses its knowledge to generate creative content aligned with the user's input.
4. *Response delivery*—The app then delivers the AI-generated content to the user in the UI. An optional feedback loop allows the user to provide comments and follow-up questions on the results. This feedback can be used to refine the response